

TRACELENS AI

Arsitektur Teknis untuk Audiensi AI Engineering

DETERMINISTIC ANALYSIS + EVIDENCE-GROUNDED AI

Versi ringkas, tanpa jargon teknis

Untuk menjelaskan konsep dan demo TraceLens AI

TraceLens AI

Arsitektur Teknis untuk Audiensi AI Engineering

Tujuan sistem

TraceLens AI adalah platform investigasi log keamanan yang mengubah file log heterogen menjadi event kanonik, timeline, correlation, finding, risk score, dan jawaban AI yang dapat ditelusuri ke raw evidence.

Prinsip terpentingnya adalah **deterministic-first**:

- parser, normalisasi timestamp, sorting, correlation, detection, dan risk scoring dilakukan oleh kode Python;
- LLM hanya memilih tool, membantu interpretasi, menyusun hipotesis, chat, dan report;
- setiap claim AI diverifikasi di backend sebelum dikirim ke investigator.

1. Technology stack

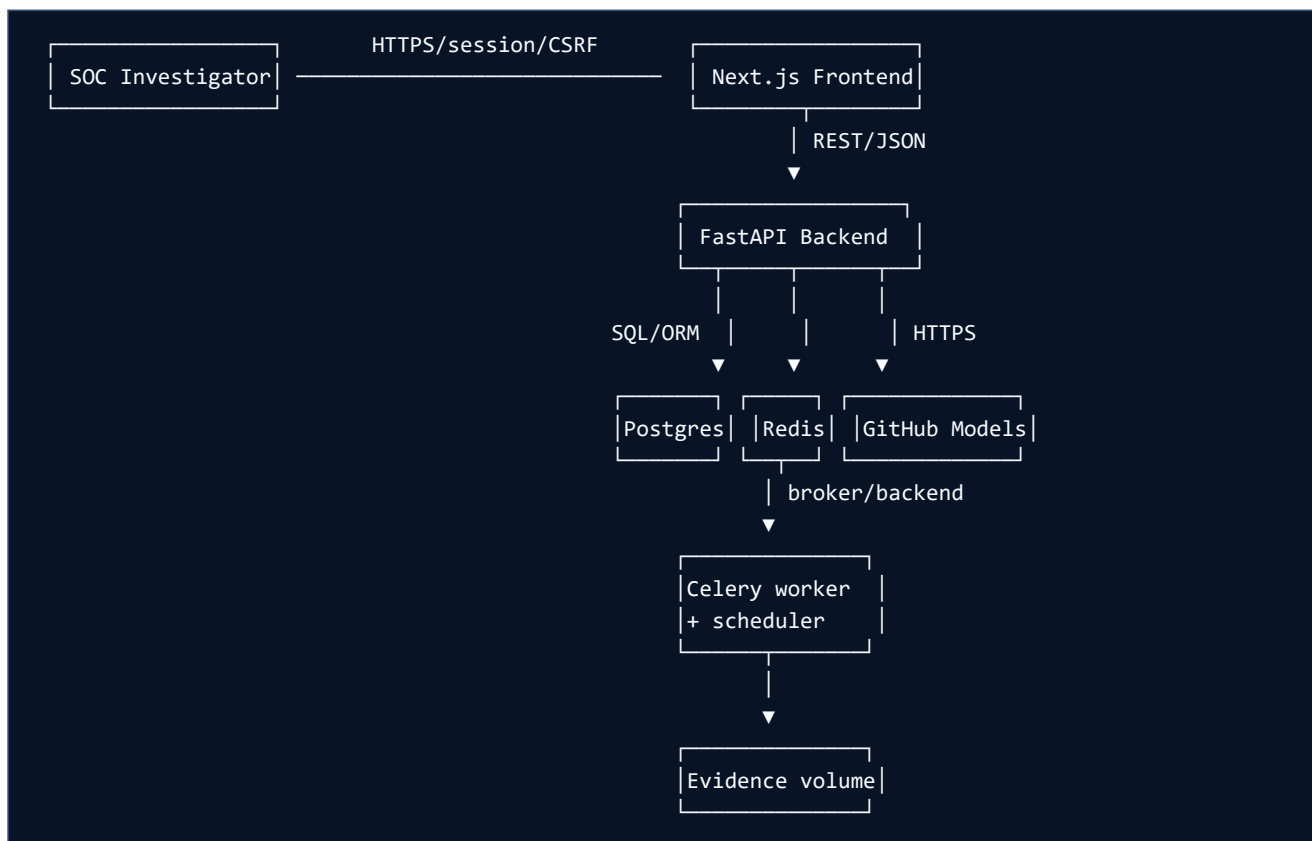
Application layer

- **Frontend**: Next.js, React, TypeScript, Tailwind CSS.
- **Backend API**: FastAPI, Python, Pydantic, SQLAlchemy.
- **Database migration**: Alembic.
- **Database**: PostgreSQL 16.
- **Queue dan cache**: Redis 7.
- **Async processing**: Celery Worker dan Celery Beat.
- **Container runtime**: Docker Compose.
- **LLM provider**: GitHub Models REST API dengan format chat-completions dan tool-calling.
- **Model aktif**: openai/gpt-4.1.
- **Testing**: pytest, parser/detection evaluation, frontend typecheck, production build.

Storage dan observability

- Docker volume untuk evidence file dalam development.
- SHA-256 untuk identitas dan integrity evidence.
- PostgreSQL untuk metadata, event, finding, audit, dan agent run.
- Prometheus metrics dan SLO rules tersedia untuk baseline observability.

2. Context diagram



3. Tanggung jawab service

Frontend

Next.js menyediakan halaman login, dashboard case, upload, timeline, event explorer, chat AI, findings, dan report. Frontend tidak melakukan analisis keamanan sendiri; frontend memanggil API dan menampilkan hasil yang sudah diverifikasi backend.

FastAPI backend

Backend mengelola authentication, authorization, case membership, upload validation, endpoint query, audit log, deterministic analysis trigger, agent orchestration, claim verification, dan export report.

PostgreSQL

PostgreSQL adalah system of record. Query child resource selalu dibatasi oleh case_id dan membership user.

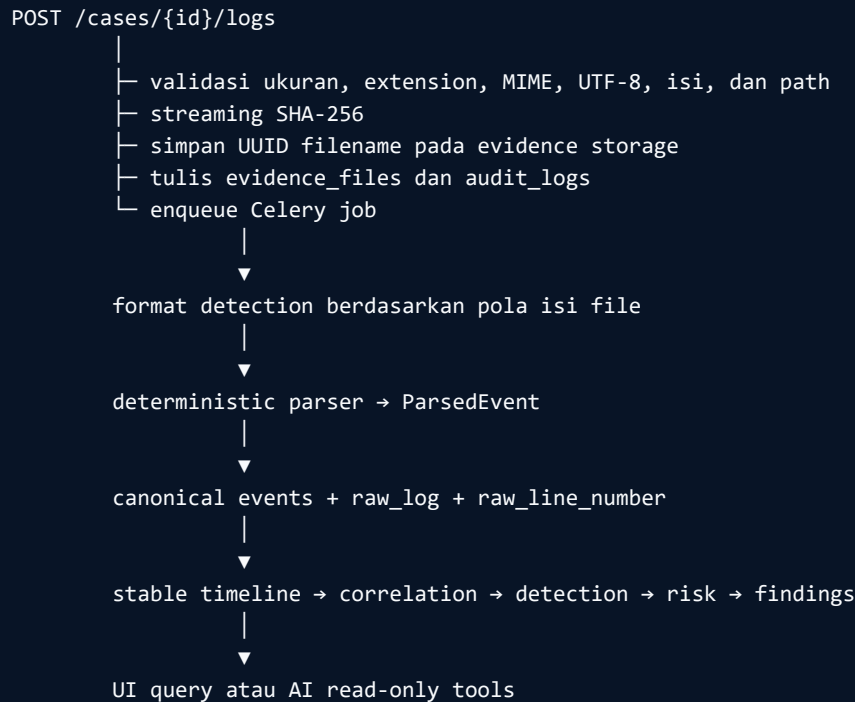
Redis dan Celery

Upload file besar tidak diproses di request HTTP. Backend menyimpan evidence dan membuat job, Redis menjadi broker, lalu Celery worker melakukan detection, parsing, normalization, correlation, detection, dan rebuild finding.

GitHub Models

Provider hanya menerima prompt dan tool context yang sudah dibatasi. LLM tidak diberi akses database langsung dan tidak membuat canonical event.

4. Data flow end-to-end



5. Deterministic analysis boundary

Parser yang tersedia meliputi Linux auth/syslog, Nginx/Apache access log, JSON application, CSV application, dan parser text/logfmt yang dikontrol konfigurasi. Format unsupported seharusnya ditolak atau diberi status experimental, bukan dianggap sebagai fakta keamanan.

Setiap event kanonik menyimpan:

- event_id dan case_id;
- timestamp_original dan timestamp_normalized;
- timezone serta provenance asumsi tahun/timezone;
- source type, host, category, action, outcome, severity;
- username, source IP, destination IP, process, file;
- raw_log dan raw_line_number;
- evidence_file_id, parser name, parser version, confidence, dan tags.

Timeline diurutkan berdasarkan timestamp normalized. Tie-breaker menggunakan line/provenance sehingga urutan dapat direproduksi.

Correlation rule-based menggunakan shared source IP, username, atau session dalam window konfigurasi. Setiap correlation memiliki correlation_reason, misalnya shared source_ip within 10 minutes.

Risk score explainable menggunakan komponen severity, frequency, privilege, correlation, dan novelty. Investigator dapat melihat breakdown, bukan hanya nilai akhir.

6. Database model

Tabel utama:

- users, user_sessions: identity dan session server-side;

- `cases`, `case_memberships`: isolation dan role;
- `evidence_files`: filename asli, internal path, hash, size, status, parser job;
- `events`: canonical security events dan provenance raw log;
- `quarantined_lines`: baris malformed pada mode quarantine;
- `correlations`: hubungan event dan correlation reason;
- `findings`: pola, risk score, breakdown, MITRE context, workflow status;
- `agent_runs`: pertanyaan, trajectory, tool budget, status, dan state;
- `audit_logs`: upload, login, chat, citation, integrity verification, export, dan perubahan workflow.

Raw event fields dijaga immutable setelah ingestion. Evidence bytes dan metadata hash menjadi dasar chain-of-custody development baseline.

7. AI agent architecture

TraceLens menggunakan **single agent dengan banyak read-only tools**, bukan multi-agent.

Tools utama:

- `search_events(case_id, filters);`
- `get_surrounding_events(event_id, window);`
- `build_timeline(case_id);`
- `correlate_entities(case_id, entity);`
- `get_raw_evidence(event_id);`
- `generate_case_summary(case_id).`

Alur chat:

```

Investigator question
  ▼
LLM gateway + system policy
  ▼
tool call dengan case_id aktif
  ▼
ToolRegistry query scoped ke case
  ▼
hasil dibungkus sebagai UNTRUSTED_DATABASE_DATA
  ▼
LLM menyusun structured claims
  ▼
claim verification gate
  ▼
jawaban yang ditampilkan ke user

```

Structured response:

```
{
  "answer": "Reconstructed only from verified claims.",
  "claims": [
    {
      "claim_id": "claim-001",
      "text": "Repeated authentication failures came from one IP.",
      "status": "fact",
      "supporting_evidence_ids": ["event-uuid"],
      "confidence": 0.95,
      "limitations": [],
      "required_additional_evidence": []
    }
  ]
}
```

Backend memeriksa evidence existence, active-case ownership, claim status, minimum support untuk inference/hypothesis, entity consistency, count consistency, limitations, dan overclaim seperti compromise atau attribution. Jawaban final dibangun ulang dari claims yang lolos.

8. AI guardrails dan konfigurasi

Konfigurasi penting:

```
GITHUB_MODELS_ENDPOINT=https://models.github.ai/inference
GITHUB_MODELS_MODEL=openai/gpt-4.1
LLM_MAX_TOOL_ROUNDS=8
LLM_MAX_TOOL_CALLS=20
LLM_MAX_TOOL_RESULT_CHARACTERS=8000
LLM_TIMEOUT_SECONDS=60
ALLOW_RAW_LOG_TO_EXTERNAL_PROVIDER=false
```

LLM_MAX_TOOL_CALLS=20 membatasi jumlah tool call per pertanyaan agar agent tidak loop dan biaya tetap terkendali. Ini bukan ukuran akurasi model.

Raw log dianggap untrusted data, diberi delimiter unik, secret redaction, truncation, dan instruksi agar model mengabaikan perintah apa pun di dalam log. Provider raw evidence sharing dimatikan secara default.

9. Security architecture

- PBKDF2-SHA256 dengan salt untuk password.
- Random session token disimpan sebagai hash.
- HttpOnly dan SameSite session cookie.
- CSRF protection pada mutation endpoint.
- Role dan membership check untuk semua case resource.
- UUID internal untuk evidence filename.
- Path traversal, absolute path, extension, MIME, size, UTF-8, dan content validation.
- SHA-256 integrity verification.
- Audit log untuk upload, chat, citation, export, dan integrity check.
- Rate limit untuk upload dan chat; login rate limiting masih menjadi hardening item.
- Docker production baseline: non-root, read-only filesystem, drop capabilities, dan no-new-privileges.

10. API surface utama

```
POST /auth/login
POST /auth/logout
GET /auth/me

POST /cases
POST /cases/{case_id}/logs
GET /cases/{case_id}/logs
GET /cases/{case_id}/jobs/{job_id}
POST /cases/{case_id}/logs/{evidence_file_id}/verify

GET /cases/{case_id}/events
GET /cases/{case_id}/events/{event_id}
GET /cases/{case_id}/timeline
GET /cases/{case_id}/entities
GET /cases/{case_id}/findings
POST /cases/{case_id}/chat
POST /cases/{case_id}/exports
```

11. Deployment topology

Development dijalankan dengan Docker Compose:

```
postgres:16-alpine
redis:7-alpine
backend: FastAPI + Alembic
worker: Celery worker
scheduler: Celery Beat
frontend: Next.js
```

Host port default:

- frontend localhost:3001;
- backend localhost:8002;
- PostgreSQL dan Redis tetap berada di network Compose.

Production overlay menambahkan hardening container. Untuk production sungguhan masih dibutuhkan TLS/WAF, managed secrets, private database/object storage, centralized immutable audit retention, backup restore drill, SLO alerting, DAST, dan independent security assessment.